

Part 1

1. Implement at least five main tables.

```
(base) yihsin@Mac ~ % mysql -u root -h 34.132.160.121 -P 3306 -p

Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 565079
Server version: 8.0.41-google (Google)

Copyright (c) 2000, 2018, Oracle and/or its affiliates. All rights reserved.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> USE CS411;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql> SHOW TABLES;
+-----+
| Tables_in_CS411 |
+-----+
| EVENT            |
| HOTEL            |
| REVIEW           |
| SAVE             |
| TRIP             |
| USER            |
| VENUE            |
+-----+
7 rows in set (0.05 sec)
```

2. DDL commands

```
CREATE TABLE USER (
    UserId VARCHAR(20) PRIMARY KEY,
    Password VARCHAR(20) NOT NULL,
    FirstName VARCHAR(255),
    LastName VARCHAR(255),
    Phone VARCHAR(15));

CREATE TABLE VENUE (
    VenueId VARCHAR(20) PRIMARY KEY,
    VenueName VARCHAR(255),
    Address VARCHAR(255),
    City VARCHAR(50),
    State VARCHAR(20),
    Zipcode CHAR(5));

CREATE TABLE EVENT (
    EventId VARCHAR(20) PRIMARY KEY,
    VenueId VARCHAR(20),
    EventName VARCHAR(255),
    URL VARCHAR(2083),
```

```
EventDate DATE,  
EventTime TIME,  
FOREIGN KEY (VenueId) REFERENCES VENUE(VenueId) ON DELETE CASCADE);
```

```
CREATE TABLE HOTEL (  
    HotelCode INT PRIMARY KEY,  
    Name VARCHAR(255),  
    State VARCHAR(20),  
    City VARCHAR(50),  
    Address VARCHAR(255),  
    Zipcode CHAR(5),  
    StarRating INT,  
    URL VARCHAR(2083));
```

```
CREATE TABLE REVIEW (  
    ReviewId INT NOT NULL,  
    HotelCode INT NOT NULL,  
    Title VARCHAR(255),  
    Content TEXT,  
    CommentDate DATE,  
    Rating DECIMAL(2,1),  
    PRIMARY KEY (ReviewId, HotelCode),  
    FOREIGN KEY (HotelCode) REFERENCES HOTEL(HotelCode) ON DELETE CASCADE);
```

```
CREATE TABLE TRIP (  
    TripId INT PRIMARY KEY,  
    UserId VARCHAR(20) NOT NULL,  
    HotelCode INT NULL,  
    EventId VARCHAR(20) NOT NULL,  
    StartDate DATE,  
    StartTime TIME,  
    UserNote TEXT,  
    Visibility BOOLEAN NOT NULL DEFAULT TRUE,  
    FOREIGN KEY (UserId) REFERENCES USER(UserId) ON DELETE CASCADE,  
    FOREIGN KEY (HotelCode) REFERENCES HOTEL(HotelCode) ON DELETE SET NULL,  
    FOREIGN KEY (EventId) REFERENCES EVENT(EventId) ON DELETE CASCADE);
```

```
CREATE TABLE SAVE (  
    UserId VARCHAR(20) NOT NULL,  
    TripId INT NOT NULL,  
    PRIMARY KEY (UserId, TripId),  
    FOREIGN KEY (UserId) REFERENCES USER(UserId) ON DELETE CASCADE,  
    FOREIGN KEY (TripId) REFERENCES TRIP(TripId) ON DELETE CASCADE);
```

3. Insert data into these tables

```
mysql> SELECT COUNT(*) FROM EVENT;
+-----+
| COUNT(*) |
+-----+
|      3919 |
+-----+
1 row in set (0.04 sec)

mysql> SELECT COUNT(*) FROM HOTEL;
+-----+
| COUNT(*) |
+-----+
|      5224 |
+-----+
1 row in set (0.04 sec)

mysql> SELECT COUNT(*) FROM REVIEW;
+-----+
| COUNT(*) |
+-----+
|    413614 |
+-----+
1 row in set (0.10 sec)

mysql> SELECT COUNT(*) FROM SAVE;
+-----+
| COUNT(*) |
+-----+
|        20 |
+-----+
1 row in set (0.03 sec)

mysql> SELECT COUNT(*) FROM TRIP;
+-----+
| COUNT(*) |
+-----+
|         4 |
+-----+
1 row in set (0.03 sec)

mysql> SELECT COUNT(*) FROM USER;
+-----+
| COUNT(*) |
+-----+
|        50 |
+-----+
1 row in set (0.03 sec)

mysql> SELECT COUNT(*) FROM VENUE;
+-----+
| COUNT(*) |
+-----+
|       251 |
+-----+
```

4. Advanced SQL queries

(1) Query for hotels in Chicago with reviews having average rating ≥ 4.5

```
SELECT h.Name, h.Address, ROUND(AVG(r.Rating),2) AS Avg_Rating
FROM HOTEL h JOIN REVIEW r USING(HotelCode)
WHERE h.city = 'Chicago'
GROUP BY HotelCode
HAVING AVG(r.Rating) >= 4.5
ORDER BY Avg_Rating DESC, h.Name
LIMIT 15;
```

```
mysql> SELECT h.Name, h.Address, ROUND(AVG(r.Rating),2) AS Avg_Rating
-> FROM HOTEL h JOIN REVIEW r USING(HotelCode)
-> WHERE h.city = 'Chicago'
-> GROUP BY HotelCode
-> HAVING AVG(r.Rating) >= 4.5
-> ORDER BY Avg_Rating DESC, h.Name
-> LIMIT 15;
```

Name	Address	Avg_Rating
Waldorf Astoria Chicago	11 East Walton	4.83
Trump International Hotel & Tower Chicago	401 N. Wabash Avenue	4.74
Royal Sonesta Chicago River North	505 North State Street	4.71
Four Seasons Hotel Chicago	120 E. Delaware Pl.	4.65
Conrad Chicago	521 North Rush Street	4.64
Hampton Majestic Chicago Theatre District	22 W Monroe St	4.64
Park Hyatt Chicago	800 N Michigan Ave	4.62
Radisson Blu Aqua Hotel	221 N. Columbus Drive	4.62
Hotel Monaco Chicago - a Kimpton Hotel	225 N Wabash	4.61
The Talbott Hotel	20 East Delaware Place	4.61
Hotel Burnham - a Kimpton Hotel	1 W Washington St	4.59
Comfort Suites	320 North Michigan Avenue	4.54
Homewood Suites by Hilton Chicago Downtown	40 East Grand Avenue	4.54
Residence Inn Chicago Downtown / River North	410 N Dearborn St	4.54

14 rows in set (10.48 sec)

*The output is less than 15 rows only 14 hotel satisfy the condition

(2) Top venues by events after 2025-01-01 and nearby hotel quality (average hotel rating in the city)

```
WITH venue_events AS (
  SELECT v.VenueId, v.VenueName, v.City, COUNT(*) AS event_count
  FROM VENUE v
  JOIN EVENT e ON e.VenueId = v.VenueId
  WHERE e.EventDate >= DATE '2025-01-01'
  GROUP BY v.VenueId, v.VenueName, v.City
),
city_hotel_avg AS (
  SELECT h.City, AVG(r.Rating) AS city_avg_hotel_rating
  FROM HOTEL h
  JOIN REVIEW r USING (HotelCode)
  GROUP BY h.City
)
SELECT ve.VenueName,
       ve.City,
       ve.event_count,
       ROUND(cha.city_avg_hotel_rating,2) AS
City_Avg_Hotel_Rating
FROM venue_events ve
JOIN city_hotel_avg cha ON cha.City = ve.City
WHERE ve.event_count > 0
ORDER BY ve.event_count DESC, ve.VenueName
LIMIT 15;
```

```
mysql> WITH venue_events AS (
-> SELECT v.VenueId, v.VenueName, v.City, COUNT(*) AS event_count
-> FROM VENUE v
-> JOIN EVENT e ON e.VenueId = v.VenueId
-> WHERE e.EventDate >= DATE '2025-01-01'
-> GROUP BY v.VenueId, v.VenueName, v.City
-> ),
-> city_hotel_avg AS (
-> SELECT h.City, AVG(r.Rating) AS city_avg_hotel_rating
-> FROM HOTEL h
-> JOIN REVIEW r USING (HotelCode)
-> GROUP BY h.City
-> )
-> SELECT ve.VenueName,
-> ve.City,
-> ve.event_count,
-> ROUND(cha.city_avg_hotel_rating,2) AS City_Avg_Hotel_Rating
-> FROM venue_events ve
-> JOIN city_hotel_avg cha ON cha.City = ve.City
-> WHERE ve.event_count > 0
-> ORDER BY ve.event_count DESC, ve.VenueName
-> LIMIT 15;
```

VenueName	City	event_count	City_Avg_Hotel_Rating
Blue Note Los Angeles	Los Angeles	167	3.76
Tractor	Seattle	79	4.12
Orchestra Hall - IL	Chicago	67	4.05
The Moroccan Lounge	Los Angeles	67	3.76
Thalia Hall	Chicago	66	4.05
Madame Lou's	Seattle	63	4.12
Walt Disney Concert Hall	Los Angeles	61	3.76
The Crocodile	Seattle	60	4.12
Brighton Music Hall presented by Citizens	Boston	58	4.05
Empty Bottle	Chicago	53	4.05
Garcia's Chicago	Chicago	53	4.05
House of Blues Chicago	Chicago	52	4.05
Paradise Rock Club presented by Citizens	Boston	51	4.05
House of Blues Dallas	Dallas	46	4.01
The Echo	Los Angeles	45	3.76

15 rows in set (32.83 sec)

(3) Hotels in New York City with the most reviews and above-average ratings

```
SELECT Name,
       City,
       COUNT(*) AS Total_Reviews,
       ROUND(AVG(Rating), 2) AS Avg_Rating
FROM HOTEL
JOIN REVIEW USING (HotelCode)
WHERE City='New York City'
GROUP BY HotelCode
HAVING AVG(Rating) >
  (SELECT AVG(hotel_ratings.hotel_avg)
   FROM
    (SELECT AVG(Rating) AS hotel_avg
     FROM HOTEL
     JOIN REVIEW USING (HotelCode)
     GROUP BY HotelCode) AS hotel_ratings)
ORDER BY Total_Reviews DESC,
```

```

        Avg_Rating DESC
LIMIT 15;
mysql> SELECT Name,
->      City,
->      COUNT(*) AS Total_Reviews,
->      ROUND(AVG(Rating), 2) AS Avg_Rating
-> FROM HOTEL
-> JOIN REVIEW USING (HotelCode)
-> WHERE City='New York City'
-> GROUP BY HotelCode
-> HAVING AVG(Rating) >
->      (SELECT AVG(hotel_ratings.hotel_avg)
->      FROM
->      (SELECT AVG(Rating) AS hotel_avg
->      FROM HOTEL
->      JOIN REVIEW USING (HotelCode)
->      GROUP BY HotelCode) AS hotel_ratings)
-> ORDER BY Total_Reviews DESC,
->      Avg_Rating DESC
-> LIMIT 15;

```

Name	City	Total_Reviews	Avg_Rating
The New Yorker Hotel	New York City	3726	4.01
Waldorf Astoria New York	New York City	3534	3.90
Affinia Manhattan	New York City	3170	4.13
Hilton New York	New York City	3004	3.76
Sofitel New York	New York City	2898	4.57
The Belvedere	New York City	2886	4.15
Grand Hyatt New York	New York City	2867	3.75
The Westin New York at Times Square	New York City	2865	4.15
New York Marriott Marquis	New York City	2839	4.17
Salisbury Hotel	New York City	2816	4.19
Novotel New York Times Square	New York City	2782	3.97
Hilton Garden Inn Times Square	New York City	2657	4.49
Pod 51	New York City	2641	4.04
Crowne Plaza Times Square Manhattan	New York City	2582	4.08
Doubletree Guest Suites Times Square	New York City	2550	4.42

15 rows in set (1 min 20.43 sec)

(4) Upcoming events in the most popular area (zipcode)

```

SELECT EventName,
       VenueName,
       EventDate,
       EventTime
FROM VENUE v
JOIN EVENT e USING (VenueId)
JOIN
  (SELECT Zipcode,
         COUNT(EventId) AS Event_Count
   FROM VENUE
   JOIN EVENT USING (VenueId)
   WHERE EventDate>='2025-11-01'
   GROUP BY Zipcode) z USING (Zipcode)
WHERE EventDate>='2025-11-01'

```

```

AND Event_Count >= ALL
    (SELECT COUNT(EventId) AS Event_Count
     FROM VENUE
     JOIN EVENT USING(VenueId)
     WHERE EventDate >= '2025-11-01'
     GROUP BY Zipcode)
ORDER BY EventDate,
         EventTime,
         EventName
LIMIT 15;

```

```

mysql> SELECT EventName,
->         VenueName,
->         EventDate,
->         EventTime
-> FROM VENUE v
-> JOIN EVENT e USING (VenueId)
-> JOIN
->   (SELECT Zipcode,
->        COUNT(EventId) AS Event_Count
->     FROM VENUE
->     JOIN EVENT USING(VenueId)
->     WHERE EventDate >= '2025-11-01'
->     GROUP BY Zipcode) z USING (Zipcode)
-> WHERE EventDate >= '2025-11-01'
-> AND Event_Count >= ALL
->   (SELECT COUNT(EventId) AS Event_Count
->    FROM VENUE
->    JOIN EVENT USING(VenueId)
->    WHERE EventDate >= '2025-11-01'
->    GROUP BY Zipcode)
-> ORDER BY EventDate,
->         EventTime,
->         EventName
-> LIMIT 15;

```

EventName	VenueName	EventDate	EventTime
The Town Hall Presents Disney And Pixar's Coco In Concert Live To Film	Town Hall	2025-11-01	14:00:00
Luther Allison Quartet	Birdland Jazz Club	2025-11-01	17:30:00
Scott Robinson's Heliotones: A Halloween Spectacular!	Birdland Theater	2025-11-01	19:00:00
The Town Hall Presents Disney And Pixar's Coco In Concert Live To Film	Town Hall	2025-11-01	19:00:00
Ron Carter's Great Big Band	Birdland Jazz Club	2025-11-01	20:30:00
Militia Vox Live	Hard Rock Cafe	2025-11-01	21:00:00
Scott Robinson's Heliotones: A Halloween Spectacular!	Birdland Theater	2025-11-01	21:30:00
Ron Carter's Great Big Band	Birdland Jazz Club	2025-11-01	22:30:00
Caelan Cardello Trio	Birdland Jazz Club	2025-11-02	17:30:00
Scott Robinson's Heliotones: A Halloween Spectacular!	Birdland Theater	2025-11-02	19:00:00
Bettye LaVette & Cyril Neville	Sony Hall	2025-11-02	20:00:00
The Afro Latin Jazz Orchestra	Birdland Jazz Club	2025-11-02	20:30:00
Scott Robinson's Heliotones: A Halloween Spectacular!	Birdland Theater	2025-11-02	21:30:00
The Afro Latin Jazz Orchestra	Birdland Jazz Club	2025-11-02	22:30:00
Vince Giordano	Birdland	2025-11-03	17:30:00

Part 2

1. Query for hotels in Chicago with reviews having average rating >= 4.5

(1) Baseline cost = 288,072

```

| -> Limit: 15 row(s) (actual time=11421..11421 rows=14 loops=1)
-> Sort: Avg_Rating DESC, h.'Name' (actual time=11421..11421 rows=14 loops=1)
-> Filter: ('avg(r.Rating)' >= 4.5) (actual time=495..11419 rows=14 loops=1)
-> Stream results (cost=288072 rows=5156) (actual time=197..11417 rows=124 loops=1)
-> Group aggregate: avg(r.Rating), avg(r.Rating) (cost=288072 rows=5156) (actual time=197..11416 rows=124 loops=1)
-> Nested loop inner join (cost=262549 rows=255230) (actual time=65.3..11390 rows=55502 loops=1)
-> Filter: (h.City = 'Chicago') (cost=609 rows=516) (actual time=5.33..61.3 rows=979 loops=1)
-> Index scan on h using PRIMARY (cost=609 rows=516) (actual time=2.99..59.4 rows=5224 loops=1)
-> Index lookup on r using HotelCode (HotelCode=h.HotelCode) (cost=459 rows=495) (actual time=0.653..11.6 rows=56.7 loops=979)

```

(2) CREATE INDEX hotel_city ON HOTEL(city); cost = 203,339

```

| -> Limit: 15 row(s) (actual time=25731..25731 rows=14 loops=1)
-> Sort: Avg_Rating DESC, h.'Name' (actual time=25731..25731 rows=14 loops=1)
-> Filter: (? ? >= 4.5) (actual time=25729..25729 rows=14 loops=1)
-> Table scan on <temporary> (actual time=25729..25729 rows=124 loops=1)
-> Aggregate using temporary table (actual time=25729..25729 rows=124 loops=1)
-> Nested loop inner join (cost=203339 rows=73407) (actual time=12..25537 rows=55502 loops=1)
-> Table scan on r (cost=68026 rows=386607) (actual time=6.27..24564 rows=413614 loops=1)
-> Filter: (h.City = 'Chicago') (cost=0.25 rows=0.19) (actual time=0.00219..0.0022 rows=0.134 loops=413614)
-> Single-row index lookup on h using PRIMARY (HotelCode=r.HotelCode) (cost=0.25 rows=1) (actual time=0.00182..0.00185 rows=1 loops=413614)

```

(3) CREATE INDEX review_rating ON REVIEW(Rating); cost = 290,140

```

| -> Limit: 15 row(s) (actual time=10778..10778 rows=14 loops=1)
-> Sort: Avg_Rating DESC, h.'Name' (actual time=10778..10778 rows=14 loops=1)
-> Filter: ('avg(r.Rating)' >= 4.5) (actual time=4829..10776 rows=14 loops=1)
-> Stream results (cost=290140 rows=5156) (actual time=208..10774 rows=124 loops=1)
-> Group aggregate: avg(r.Rating), avg(r.Rating) (cost=290140 rows=5156) (actual time=208..10771 rows=124 loops=1)
-> Nested loop inner join (cost=264617 rows=255230) (actual time=69.4..10743 rows=55502 loops=1)
-> Filter: (h.City = 'Chicago') (cost=576 rows=516) (actual time=7.47..57 rows=979 loops=1)
-> Index scan on h using PRIMARY (cost=576 rows=516) (actual time=5.1..55.3 rows=5224 loops=1)
-> Index lookup on r using HotelCode (HotelCode=h.HotelCode) (cost=463 rows=495) (actual time=0.6..10.9 rows=56.7 loops=979)

```

(4) CREATE INDEX hotel_name ON HOTEL(Name); cost = 287,738

```

| -> Limit: 15 row(s) (actual time=10590..10590 rows=14 loops=1)
-> Sort: Avg_Rating DESC, h.'Name' (actual time=10590..10590 rows=14 loops=1)
-> Filter: ('avg(r.Rating)' >= 4.5) (actual time=5169..10588 rows=14 loops=1)
-> Stream results (cost=287738 rows=5156) (actual time=153..10587 rows=124 loops=1)
-> Group aggregate: avg(r.Rating), avg(r.Rating) (cost=287738 rows=5156) (actual time=153..10585 rows=124 loops=1)
-> Nested loop inner join (cost=262215 rows=255230) (actual time=51.4..10560 rows=55502 loops=1)
-> Filter: (h.City = 'Chicago') (cost=606 rows=516) (actual time=3.92..75.5 rows=979 loops=1)
-> Index scan on h using PRIMARY (cost=606 rows=516) (actual time=2.03..73.5 rows=5224 loops=1)
-> Index lookup on r using HotelCode (HotelCode=h.HotelCode) (cost=458 rows=495) (actual time=0.582..10.7 rows=56.7 loops=979)

```

Analysis:

The baseline query cost was 288,072. After creating the HOTEL(City) index, the query cost significantly dropped to 203,339. This reduction demonstrates a considerable boost in query efficiency.

In contrast, other indexing strategies, such as indexing the REVIEW(Rating) column, had a negative impact on performance, causing the query cost to increase to 290,140. Similarly, creating an index on HOTEL(Name) led to a marginal change in cost (to 287,738), showing little to no significant effect on query performance.

The superiority of the HOTEL(City) index lies in its high selectivity and its strategic placement within the query. The index directly optimizes the critical WHERE condition (h.City = 'Chicago'), enabling the query optimizer to choose a more efficient execution plan. This results in a significant performance gain, particularly for a query like this, where filtering by city is a key factor in reducing the data set size. Therefore, our index design selection would be HOTEL(City).

2. Top venues by upcoming events + nearby hotel quality

(1) Baseline cost = 96,654


```

1 -> Limit: 15 row(s) (cost=96654 rows=0) (actual time=27127..27127 rows=15 loops=1)
   -> Nested loop inner join (cost=96654 rows=0) (actual time=27127..27127 rows=15 loops=1)
       -> Sort: ve.event_count DESC, ve.VenueName (cost=2.6..2.6 rows=0) (actual time=53.4..53.4 rows=22 loops=1)
       -> Filter: (ve.City is not null) (cost=2.5..2.5 rows=0) (actual time=51.3..51.4 rows=250 loops=1)
       -> Table scan on ve (cost=2.5..2.5 rows=0) (actual time=51.3..51.4 rows=250 loops=1)
       -> Materialize CTE venue_events (cost=0..0 rows=0) (actual time=51.3..51.3 rows=250 loops=1)
       -> Filter: (event_count > 0) (actual time=51..51.1 rows=250 loops=1)
       -> Table scan on <temporary> (actual time=51..51.1 rows=250 loops=1)
       -> Aggregate using temporary table (actual time=51..51 rows=250 loops=1)
       -> Nested loop inner join (cost=746 rows=1110) (actual time=2.44..44.2 rows=3919 loops=1)
           -> Filter: ((e.EventDate >= DATE'2025-01-01') and (e.VenueId is not null)) (cost=357 rows=1110) (actual time=0.245..32.3 rows=3919 loops=1)
           -> Table scan on e (cost=357 rows=3330) (actual time=0.206..31.4 rows=3919 loops=1)
           -> Single-row index lookup on v using PRIMARY (VenueId=e.VenueId) (cost=0.25 rows=1) (actual time=0.0028..0.00283 rows=1 loops=3919)
       -> Index lookup on cha using <auto_key> (City=ve.City) (cost=0.25..87.2 rows=349) (actual time=1231..1231 rows=0.682 loops=22)
       -> Materialize CTE city_hotel_avg (cost=0..0 rows=0) (actual time=27074..27074 rows=6 loops=1)
       -> Table scan on <temporary> (actual time=27073..27073 rows=6 loops=1)
       -> Aggregate using temporary table (actual time=27073..27073 rows=6 loops=1)
       -> Nested loop inner join (cost=307024 rows=386607) (actual time=12.1..26623 rows=413614 loops=1)
           -> Table scan on r (cost=68507 rows=386607) (actual time=11.4..25828 rows=413614 loops=1)
           -> Single-row index lookup on h using PRIMARY (HotelCode=r.HotelCode) (cost=0.517 rows=1) (actual time=0.00166..0.0017 rows=1 loops=413614)

```

(2) CREATE INDEX event_date_idx ON EVENT(EventDate); cost =96,654

```

1 -> Limit: 15 row(s) (cost=96654 rows=0) (actual time=25200..25200 rows=15 loops=1)
   -> Nested loop inner join (cost=96654 rows=0) (actual time=25200..25200 rows=15 loops=1)
       -> Sort: ve.event_count DESC, ve.VenueName (cost=2.6..2.6 rows=0) (actual time=39.5..39.5 rows=22 loops=1)
       -> Filter: (ve.City is not null) (cost=2.5..2.5 rows=0) (actual time=38.4..38.5 rows=250 loops=1)
       -> Table scan on ve (cost=2.5..2.5 rows=0) (actual time=38.4..38.5 rows=250 loops=1)
       -> Materialize CTE venue_events (cost=0..0 rows=0) (actual time=38.4..38.4 rows=250 loops=1)
       -> Filter: (event_count > 0) (actual time=38.2..38.3 rows=250 loops=1)
       -> Table scan on <temporary> (actual time=38.2..38.3 rows=250 loops=1)
       -> Aggregate using temporary table (actual time=38.2..38.2 rows=250 loops=1)
       -> Nested loop inner join (cost=1196 rows=3343) (actual time=4.82..32.4 rows=3919 loops=1)
           -> Table scan on v (cost=25.9 rows=251) (actual time=3.13..3.45 rows=251 loops=1)
           -> Filter: (e.EventDate >= DATE'2025-01-01') (cost=3.34 rows=13.3) (actual time=0.0233..0.114 rows=15.6 loops=251)
           -> Index lookup on e using event_venue (VenueId=v.VenueId) (cost=3.34 rows=13.3) (actual time=0.023..0.112 rows=15.6 loops=251)
       -> Index lookup on cha using <auto_key> (City=ve.City) (cost=0.25..28.9 rows=116) (actual time=1144..1144 rows=0.682 loops=22)
       -> Materialize CTE city_hotel_avg (cost=0..0 rows=0) (actual time=25160..25160 rows=6 loops=1)
       -> Table scan on <temporary> (actual time=25160..25160 rows=6 loops=1)
       -> Aggregate using temporary table (actual time=25160..25160 rows=6 loops=1)
       -> Nested loop inner join (cost=306869 rows=386607) (actual time=7.64..24732 rows=413614 loops=1)
           -> Table scan on r (cost=68352 rows=386607) (actual time=7.61..23968 rows=413614 loops=1)
           -> Single-row index lookup on h using PRIMARY (HotelCode=r.HotelCode) (cost=0.517 rows=1) (actual time=0.0016..0.00163 rows=1 loops=413614)

```

(3) CREATE INDEX hotel_city_idx ON HOTEL(City); cost=96,654

```

1 -> Limit: 15 row(s) (cost=96654 rows=0) (actual time=25067..25067 rows=15 loops=1)
   -> Nested loop inner join (cost=96654 rows=0) (actual time=25067..25067 rows=15 loops=1)
       -> Sort: ve.event_count DESC, ve.VenueName (cost=2.6..2.6 rows=0) (actual time=14.6..14.6 rows=22 loops=1)
       -> Filter: (ve.City is not null) (cost=2.5..2.5 rows=0) (actual time=14.4..14.4 rows=250 loops=1)
       -> Table scan on ve (cost=2.5..2.5 rows=0) (actual time=14.4..14.4 rows=250 loops=1)
       -> Materialize CTE venue_events (cost=0..0 rows=0) (actual time=14.4..14.4 rows=250 loops=1)
       -> Filter: (event_count > 0) (actual time=14.2..14.2 rows=250 loops=1)
       -> Table scan on <temporary> (actual time=14.1..14.2 rows=250 loops=1)
       -> Aggregate using temporary table (actual time=14.1..14.1 rows=250 loops=1)
       -> Nested loop inner join (cost=746 rows=1110) (actual time=0.086..8.6 rows=3919 loops=1)
           -> Filter: ((e.EventDate >= DATE'2025-01-01') and (e.VenueId is not null)) (cost=357 rows=1110) (actual time=0.0577..2.5 rows=3919 loops=1)
           -> Table scan on e (cost=357 rows=3330) (actual time=0.0519..1.83 rows=3919 loops=1)
           -> Single-row index lookup on v using PRIMARY (VenueId=e.VenueId) (cost=0.25 rows=1) (actual time=0.00137..0.00139 rows=1 loops=3919)
       -> Index lookup on cha using <auto_key> (City=ve.City) (cost=0.25..87.2 rows=349) (actual time=1139..1139 rows=0.682 loops=22)
       -> Materialize CTE city_hotel_avg (cost=0..0 rows=0) (actual time=25053..25053 rows=6 loops=1)
       -> Table scan on <temporary> (actual time=25053..25053 rows=6 loops=1)
       -> Aggregate using temporary table (actual time=25053..25053 rows=6 loops=1)
       -> Nested loop inner join (cost=203685 rows=386607) (actual time=8.24..24626 rows=413614 loops=1)
           -> Table scan on r (cost=68373 rows=386607) (actual time=8.21..23876 rows=413614 loops=1)
           -> Single-row index lookup on h using PRIMARY (HotelCode=r.HotelCode) (cost=0.25 rows=1) (actual time=0.00157..0.0016 rows=1 loops=413614)

```

(4) CREATE INDEX idx_venue_city ON VENUE (City); cost=96,654

```

1 -> Nested loop inner join (cost=96654 rows=0) (actual time=24366..24366 rows=190 loops=1)
   -> Sort: ve.event_count DESC, ve.VenueName (cost=2.6..2.6 rows=0) (actual time=29.2..29.2 rows=250 loops=1)
   -> Filter: (ve.City is not null) (cost=2.5..2.5 rows=0) (actual time=27.5..27.5 rows=250 loops=1)
   -> Table scan on ve (cost=2.5..2.5 rows=0) (actual time=27.5..27.5 rows=250 loops=1)
   -> Materialize CTE venue_events (cost=0..0 rows=0) (actual time=27.5..27.5 rows=250 loops=1)
   -> Filter: (event_count > 0) (actual time=27..27.1 rows=250 loops=1)
   -> Table scan on <temporary> (actual time=27..27 rows=250 loops=1)
   -> Aggregate using temporary table (actual time=27..27 rows=250 loops=1)
   -> Nested loop inner join (cost=746 rows=1110) (actual time=0.557..16.8 rows=3919 loops=1)
       -> Filter: ((e.EventDate >= DATE'2025-01-01') and (e.VenueId is not null)) (cost=357 rows=1110) (actual time=0.519..8.8 rows=3919 loops=1)
       -> Table scan on e (cost=357 rows=3330) (actual time=0.378..7.64 rows=3919 loops=1)
       -> Single-row index lookup on v using PRIMARY (VenueId=e.VenueId) (cost=0.25 rows=1) (actual time=0.00177..0.00181 rows=1 loops=3919)
   -> Index lookup on cha using <auto_key> (City=ve.City) (cost=0.25..87.2 rows=349) (actual time=97.3..97.3 rows=0.76 loops=250)
   -> Materialize CTE city_hotel_avg (cost=0..0 rows=0) (actual time=24337..24337 rows=6 loops=1)
   -> Table scan on <temporary> (actual time=24337..24337 rows=6 loops=1)
   -> Aggregate using temporary table (actual time=24337..24337 rows=6 loops=1)
   -> Nested loop inner join (cost=306950 rows=386607) (actual time=14.2..23909 rows=413614 loops=1)
       -> Table scan on r (cost=68433 rows=386607) (actual time=14..23139 rows=413614 loops=1)
       -> Single-row index lookup on h using PRIMARY (HotelCode=r.HotelCode) (cost=0.517 rows=1) (actual time=0.0016..0.00163 rows=1 loops=413614)

```

Analysis:

The baseline query cost was 96,654. After creating the EVENT(EventDate) index, the query cost remained 96,654, showing no measurable improvement. Similarly, creating indexes on HOTEL(City) and VENUE(City) produced the same total cost, again showing no significant change in performance.

The lack of variation across all three index tests suggests that our indexing did not substantially impact the optimizer's chosen plan. This is likely because the query's main workload comes from aggregating and materializing the two CTEs (venue_events and city_hotel_avg), rather than filtering large datasets or direct lookups. Since MySQL must still build temporary tables to compute grouped values for both venue_events and city_hotel_avg, the indexes could not reduce that overhead. Another possible reason is that in the current dataset, most event records are from 2025 and 2026, so the date filtering efficiency remains narrow. However, as data accumulates over multiple years, the effectiveness of filtering date will likely increase.

Although the performance metrics remained consistent, the EVENT(EventDate) index is still the most strategically valuable for our index design. This index provides support for filtering, and as the number of event records increases, it should enable the optimizer to locate events more efficiently.

3. Hotels in New York City with the most reviews and above-average ratings

(1) Baseline, cost = 311,108

```
| -> Limit: 15 row(s) (actual time=79019..79019 rows=15 loops=1)
  -> Sort: Total_Reviews DESC, Avg_Rating DESC (actual time=79019..79019 rows=15 loops=1)
  -> Filter: ('avg(REVIEW.Rating)' > (select #2)) (actual time=32989..79015 rows=266 loops=1)
  -> Stream results (cost=311108 rows=5156) (actual time=2502..48526 rows=360 loops=1)
    -> Group aggregate: avg(REVIEW.Rating), avg(REVIEW.Rating), count(0) (cost=311108 rows=5156) (actual time=2502..48520 rows=360 loops=1)
      -> Nested loop inner join (cost=284845 rows=262628) (actual time=97.9..48406 rows=232252 loops=1)
        -> Filter: (HOTEL.City = 'New York City') (cost=613 rows=516) (actual time=15.9..84.4 rows=383 loops=1)
        -> Index scan on HOTEL using PRIMARY (cost=613 rows=516) (actual time=9.06..80.3 rows=5224 loops=1)
        -> Index lookup on REVIEW using review_hotel_idx (HotelCode=HOTEL.HotelCode) (cost=500 rows=509) (actual time=5.23..126 rows=606 loops=383)
      -> Select #2 (subquery in condition; run only once)
        -> Aggregate: avg(hotel_ratings.hotel_avg) (cost=2.5..2.5 rows=1) (actual time=30487..30487 rows=1 loops=1)
        -> Table scan on hotel_ratings (cost=2.5..2.5 rows=0) (actual time=30487..30487 rows=1014 loops=1)
        -> Materialize (cost=0..0 rows=0) (actual time=30487..30487 rows=1014 loops=1)
        -> Table scan on <temporary> (actual time=30485..30485 rows=1014 loops=1)
        -> Aggregate using temporary table (actual time=30485..30485 rows=1014 loops=1)
        -> Nested loop inner join (cost=495166 rows=386607) (actual time=7.19..30122 rows=413614 loops=1)
        -> Table scan on REVIEW (cost=69899 rows=386607) (actual time=7.16..29199 rows=413614 loops=1)
        -> Single-row covering index lookup on HOTEL using PRIMARY (HotelCode=REVIEW.HotelCode) (cost=1 rows=1) (actual time=0.00192..0.00195 rows=1 loops=413614)
```

(2) CREATE INDEX review_rating_idx ON REVIEW(Rating); cost = 310,624

```

| -> Limit: 15 row(s) (actual time=51943..51943 rows=15 loops=1)
    -> Sort: Total_Reviews DESC, Avg_Rating DESC (actual time=51943..51943 rows=15 loops=1)
    -> Filter: ('avg(REVIEW.Rating)' > (select #2)) (actual time=3900..51938 rows=266 loops=1)
    -> Stream results (cost=310624 rows=5156) (actual time=2413..50449 rows=360 loops=1)
    -> Group aggregate: avg(REVIEW.Rating), avg(REVIEW.Rating), count(0) (cost=310624 rows=5156) (actual time=2413..50443 rows=360 loops=1)
    -> Nested loop inner join (cost=284362 rows=262628) (actual time=76.4..50326 rows=232252 loops=1)
    -> Filter: (HOTEL.City = 'New York City') (cost=613 rows=516) (actual time=11.2..80 rows=383 loops=1)
    -> Index scan on HOTEL using PRIMARY (cost=613 rows=516) (actual time=5.88..76.7 rows=5224 loops=1)
    -> Index lookup on REVIEW using review_hotel_idx (HotelCode=HOTEL.HotelCode) (cost=499 rows=509) (actual time=5.42..131 rows=606
loops=383)
    -> Select #2 (subquery in condition; run only once)
    -> Aggregate: avg(hotel_ratings.hotel_avg) (cost=2.5..2.5 rows=1) (actual time=1487..1487 rows=1 loops=1)
    -> Table scan on hotel_ratings (cost=2.5..2.5 rows=0) (actual time=1487..1487 rows=1014 loops=1)
    -> Materialize (cost=0..0 rows=0) (actual time=1487..1487 rows=1014 loops=1)
    -> Table scan on <temporary> (actual time=1486..1487 rows=1014 loops=1)
    -> Aggregate using temporary table (actual time=1486..1486 rows=1014 loops=1)
    -> Nested loop inner join (cost=495108 rows=386607) (actual time=7.05..1256 rows=413614 loops=1)
    -> Covering index scan on REVIEW using review_rating_idx (cost=69840 rows=386607) (actual time=7.01..579 rows=413
614 loops=1)
    -> Single-row covering index lookup on HOTEL using PRIMARY (HotelCode=REVIEW.HotelCode) (cost=1 rows=1) (actual t
ime=0.00136..0.0014 rows=1 loops=413614)
|

```

(3) CREATE INDEX hotel_city ON HOTEL(City); cost=233,250

```

| -> Limit: 15 row(s) (actual time=80683..80683 rows=15 loops=1)
    -> Sort: Total_Reviews DESC, Avg_Rating DESC (actual time=80683..80683 rows=15 loops=1)
    -> Filter: ('avg(REVIEW.Rating)' > (select #2)) (actual time=32594..80680 rows=266 loops=1)
    -> Stream results (cost=233250 rows=5156) (actual time=2386..50470 rows=360 loops=1)
    -> Group aggregate: avg(REVIEW.Rating), avg(REVIEW.Rating), count(0) (cost=233250 rows=5156) (actual time=2386..50464 rows=360 loops=1)
    -> Nested loop inner join (cost=213741 rows=195086) (actual time=61.3..50347 rows=232252 loops=1)
    -> Index lookup on HOTEL using hotel_city (City='New York City') (cost=192 rows=383) (actual time=0.185..42.6 rows=383 loops=1)
    -> Index lookup on REVIEW using review_hotel_idx (HotelCode=HOTEL.HotelCode) (cost=507 rows=509) (actual time=5.95..131 rows=606
loops=383)
    -> Select #2 (subquery in condition; run only once)
    -> Aggregate: avg(hotel_ratings.hotel_avg) (cost=2.5..2.5 rows=1) (actual time=30204..30204 rows=1 loops=1)
    -> Table scan on hotel_ratings (cost=2.5..2.5 rows=0) (actual time=30204..30204 rows=1014 loops=1)
    -> Materialize (cost=0..0 rows=0) (actual time=30204..30204 rows=1014 loops=1)
    -> Table scan on <temporary> (actual time=30203..30203 rows=1014 loops=1)
    -> Aggregate using temporary table (actual time=30203..30203 rows=1014 loops=1)
    -> Nested loop inner join (cost=313724 rows=386607) (actual time=10.7..29825 rows=413614 loops=1)
    -> Table scan on REVIEW (cost=70292 rows=386607) (actual time=8.13..28862 rows=413614 loops=1)
    -> Single-row covering index lookup on HOTEL using PRIMARY (HotelCode=REVIEW.HotelCode) (cost=0.53 rows=1) (actua
l time=0.002..0.00204 rows=1 loops=413614)
|

```

(4) CREATE INDEX idx_city_hotelcode ON HOTEL (City, HotelCode); cost = 205,124

```

| -> Limit: 15 row(s) (actual time=61122..61122 rows=15 loops=1)
    -> Sort: Total_Reviews DESC, Avg_Rating DESC (actual time=61122..61122 rows=15 loops=1)
    -> Filter: (??? > (select #2)) (actual time=61122..61122 rows=266 loops=1)
    -> Table scan on <temporary> (actual time=31394..31394 rows=360 loops=1)
    -> Aggregate using temporary table (actual time=31394..31394 rows=360 loops=1)
    -> Nested loop inner join (cost=205124 rows=28718) (actual time=32.3..30183 rows=232252 loops=1)
    -> Table scan on REVIEW (cost=69812 rows=386607) (actual time=8.4..29063 rows=413614 loops=1)
    -> Filter: (HOTEL.City = 'New York City') (cost=0.25 rows=0.0743) (actual time=0.00241..0.00248 rows=0.562 loops=413614)
    -> Single-row index lookup on HOTEL using PRIMARY (HotelCode=REVIEW.HotelCode) (cost=0.25 rows=1) (actual time=0.00194..0.001
98 rows=1 loops=413614)
    -> Select #2 (subquery in condition; run only once)
    -> Aggregate: avg(hotel_ratings.hotel_avg) (cost=2.5..2.5 rows=1) (actual time=29728..29728 rows=1 loops=1)
    -> Table scan on hotel_ratings (cost=2.5..2.5 rows=0) (actual time=29728..29728 rows=1014 loops=1)
    -> Materialize (cost=0..0 rows=0) (actual time=29728..29728 rows=1014 loops=1)
    -> Table scan on <temporary> (actual time=29728..29728 rows=1014 loops=1)
    -> Aggregate using temporary table (actual time=29728..29728 rows=1014 loops=1)
    -> Nested loop inner join (cost=205124 rows=386607) (actual time=8.18..29357 rows=413614 loops=1)
    -> Table scan on REVIEW (cost=69812 rows=386607) (actual time=8.15..28466 rows=413614 loops=1)
    -> Single-row covering index lookup on HOTEL using PRIMARY (HotelCode=REVIEW.HotelCode) (cost=0.25 rows=1) (actua
l time=0.00183..0.00187 rows=1 loops=413614)
|

```

Analysis:

The baseline query cost was 311,108. After creating the REVIEW(Rating) index produced a cost of 310,624, which did not enhance much performance. In contrast, creating the HOTEL(City) index reduced the query cost to 233,250, and the composite index HOTEL(City, HotelCode) index reduced the query cost to 205,124.

further reduced it to 205,124. These results suggest that location-based and join-oriented indexes provided a more optimal access path among all tested designs.

The limited effect of the REVIEW(Rating) index is likely because the query involves multiple aggregation on ratings, rather than by the join lookups themselves. The improvement from the HOTEL(City) index reflects that allowing MySQL to quickly isolate hotels located in New York City can significantly reduce the number of rows involved and the cost. Additionally, the composite HOTEL(City, HotelCode) index further supports the join on HotelCode, and generates a more efficient access path.

The final index design chosen for Query 3 is HOTEL(City, HotelCode) because it produced the largest drop in cost and yielded the most efficient plan.

4. Upcoming Events in the Most Popular Area (Zipcode)

(1) Baseline, cost = 7,122

```

1 -> Limit: 15 row(s) (cost=7122 rows=15) (actual time=28.7..29.5 rows=15 loops=1)
    -> Nested loop inner join (cost=7122 rows=22383) (actual time=28.7..29.5 rows=15 loops=1)
        -> Nested loop inner join (cost=968 rows=3330) (actual time=10.4..10.9 rows=170 loops=1)
            -> Sort: e.EventDate, e.EventTime, e.EventName (cost=357 rows=3330) (actual time=10.3..10.4 rows=170 loops=1)
            -> Filter: ((e.EventDate >= DATE'2025-11-01') and (e.VenueId is not null)) (cost=357 rows=3330) (actual time=0.085..2.83 rows=3597 loops=1)
                -> Table scan on e (cost=357 rows=3330) (actual time=0.0748..2.32 rows=3919 loops=1)
            -> Filter: (v.Zipcode is not null) (cost=0.25 rows=1) (actual time=0.00256..0.00265 rows=1 loops=170)
                -> Single-row index lookup on v using PRIMARY (VenueId=e.VenueId) (cost=0.25 rows=1) (actual time=0.00241..0.00244 rows=1 loops=170)
        -> Filter: <not>((z.Event_Count < <max>(select #3))) (cost=0.375..2.52 rows=6.72) (actual time=0.109..0.109 rows=0.0882 loops=170)
            -> Index lookup on z using <auto_key> (Zipcode=v.Zipcode) (cost=0.25..2.52 rows=10.1) (actual time=0.0507..0.051 rows=1 loops=170)
            -> Materialize (cost=0..0 rows=0) (actual time=8.39..8.39 rows=96 loops=1)
                -> Table scan on <temporary> (actual time=8.25..8.26 rows=96 loops=1)
                    -> Aggregate using temporary table (actual time=8.25..8.25 rows=96 loops=1)
                        -> Nested loop inner join (cost=746 rows=1110) (actual time=0.046..6.99 rows=3597 loops=1)
                            -> Filter: (('EVENT'.EventDate >= DATE'2025-11-01') and ('EVENT'.VenueId is not null)) (cost=357 rows=1110) (actual time=0.0353..2 rows=3597 loops=1)
                                -> Table scan on EVENT (cost=357 rows=3330) (actual time=0.0299..1.54 rows=3919 loops=1)
                                    -> Single-row index lookup on VENUE using PRIMARY (VenueId='EVENT'.VenueId) (cost=0.25 rows=1) (actual time=0.0012..0.00123 rows=1 loops=3597)
                            -> Select #3 (subquery in condition; run only once)
                                -> Table scan on <temporary> (actual time=9.82..9.83 rows=96 loops=1)
                                    -> Aggregate using temporary table (actual time=9.81..9.81 rows=96 loops=1)
                                        -> Nested loop inner join (cost=746 rows=1110) (actual time=0.0728..8.09 rows=3597 loops=1)
                                            -> Filter: (('EVENT'.EventDate >= DATE'2025-11-01') and ('EVENT'.VenueId is not null)) (cost=357 rows=1110) (actual time=0.0534..2.42 rows=3597 loops=1)
                                                -> Table scan on EVENT (cost=357 rows=3330) (actual time=0.0501..1.82 rows=3919 loops=1)
                                                    -> Single-row index lookup on VENUE using PRIMARY (VenueId='EVENT'.VenueId) (cost=0.25 rows=1) (actual time=0.00138..0.00141 rows=1 loops=3597)

```

(2) CREATE INDEX event_date_idx ON EVENT(EventDate); cost = 17,046

```

1 -> Limit: 15 row(s) (cost=17046 rows=15) (actual time=49.2..49.9 rows=15 loops=1)
    -> Nested loop inner join (cost=17046 rows=29569) (actual time=49.2..49.9 rows=15 loops=1)
        -> Nested loop inner join (cost=1523 rows=3330) (actual time=11..11.4 rows=170 loops=1)
            -> Sort: e.EventDate, e.EventTime, e.EventName (cost=357 rows=3330) (actual time=10.9..10.9 rows=170 loops=1)
            -> Filter: ((e.EventDate >= DATE'2025-11-01') and (e.VenueId is not null)) (cost=357 rows=3330) (actual time=0.453..2.98 rows=3597 loops=1)
                -> Table scan on e (cost=357 rows=3330) (actual time=0.447..2.49 rows=3919 loops=1)
            -> Filter: (v.Zipcode is not null) (cost=0.25 rows=1) (actual time=0.00235..0.00243 rows=1 loops=170)
                -> Single-row index lookup on v using PRIMARY (VenueId=e.VenueId) (cost=0.25 rows=1) (actual time=0.00221..0.00224 rows=1 loops=170)
        -> Filter: <not>((z.Event_Count < <max>(select #3))) (cost=0.375..3.33 rows=8.88) (actual time=0.226..0.226 rows=0.0882 loops=170)
            -> Index lookup on z using <auto_key> (Zipcode=v.Zipcode) (cost=0.25..3.33 rows=13.3) (actual time=0.156..0.156 rows=1 loops=170)
            -> Materialize (cost=0..0 rows=0) (actual time=26.3..26.3 rows=96 loops=1)
                -> Table scan on <temporary> (actual time=26.2..26.2 rows=96 loops=1)
                    -> Aggregate using temporary table (actual time=26.2..26.2 rows=96 loops=1)
                        -> Nested loop inner join (cost=1196 rows=3343) (actual time=3.09..24.7 rows=3597 loops=1)
                            -> Table scan on VENUE (cost=25.9 rows=251) (actual time=0.424..0.597 rows=251 loops=1)
                                -> Filter: ('EVENT'.EventDate >= DATE'2025-11-01') (cost=3.34 rows=13.3) (actual time=0.0231..0.0949 rows=14.3 loops=251)
                                    -> Index lookup on EVENT using event_venue (VenueId=VENUE.VenueId) (cost=3.34 rows=13.3) (actual time=0.0228..0.0932 rows=15.6 loops=251)
                            -> Select #3 (subquery in condition; run only once)
                                -> Table scan on <temporary> (actual time=11.8..11.8 rows=96 loops=1)
                                    -> Aggregate using temporary table (actual time=11.8..11.8 rows=96 loops=1)
                                        -> Nested loop inner join (cost=1196 rows=3343) (actual time=0.412..10.5 rows=3597 loops=1)
                                            -> Table scan on VENUE (cost=25.9 rows=251) (actual time=0.351..0.455 rows=251 loops=1)
                                                -> Filter: ('EVENT'.EventDate >= DATE'2025-11-01') (cost=3.34 rows=13.3) (actual time=0.00797..0.0387 rows=14.3 loops=251)
                                                    -> Index lookup on EVENT using event_venue (VenueId=VENUE.VenueId) (cost=3.34 rows=13.3) (actual time=0.00777..0.0371 rows=15.6 loops=251)

```

(3) CREATE INDEX event_date_idx ON EVENT(EventTime); cost = 7,122


```

1 -> Limit: 15 row(s) (cost=7122 rows=15) (actual time=23.6..24.3 rows=15 loops=1)
   -> Nested loop inner join (cost=7122 rows=22383) (actual time=23.6..24.2 rows=15 loops=1)
     -> Nested loop inner join (cost=968 rows=3330) (actual time=5.75..6.17 rows=170 loops=1)
       -> Sort: e.EventDate, e.EventTime, e.EventName (cost=357 rows=3330) (actual time=5.72..5.74 rows=170 loops=1)
       -> Filter: ((e.EventDate >= DATE'2025-11-01') and (e.VenueId is not null)) (cost=357 rows=3330) (actual time=0.0967..2.29 rows=3597 loops=1)
       -> Table scan on e (cost=357 rows=3330) (actual time=0.0418..1.8 rows=3919 loops=1)
       -> Filter: (v.Zipcode is not null) (cost=0.25 rows=1) (actual time=0.00226..0.00235 rows=1 loops=170)
     -> Single-row index lookup on v using PRIMARY (VenueId=e.VenueId) (cost=0.25 rows=1) (actual time=0.00212..0.00215 rows=1 loops=170)
   -> Filter: <not>((z.Event_Count < <max>(select #3))) (cost=0.375..2.52 rows=6.72) (actual time=0.106..0.106 rows=0.0882 loops=170)
   -> Index lookup on z using <auto_key0> (Zipcode=v.Zipcode) (cost=0.25..2.52 rows=10.1) (actual time=0.0522..0.0525 rows=1 loops=170)
   -> Materialize (cost=0..0 rows=0) (actual time=8.7..8.7 rows=96 loops=1)
   -> Table scan on <temporary> (actual time=8.59..8.6 rows=96 loops=1)
   -> Aggregate using temporary table (actual time=8.59..8.59 rows=96 loops=1)
     -> Nested loop inner join (cost=746 rows=1110) (actual time=0.0547..7.3 rows=3597 loops=1)
       -> Filter: (('EVENT'.EventDate >= DATE'2025-11-01') and ('EVENT'.VenueId is not null)) (cost=357 rows=1110) (actual time=0.0464..2.02 rows=3597 loops=1)
       -> Table scan on EVENT (cost=357 rows=3330) (actual time=0.0428..1.54 rows=3919 loops=1)
       -> Single-row index lookup on VENUE using PRIMARY (VenueId='EVENT'.VenueId) (cost=0.25 rows=1) (actual time=0.00129..0.00131 rows=1 loops=3597)
     -> Select #3 (subquery in condition; run only once)
     -> Table scan on <temporary> (actual time=9.04..9.05 rows=96 loops=1)
     -> Aggregate using temporary table (actual time=9.04..9.04 rows=96 loops=1)
     -> Nested loop inner join (cost=746 rows=1110) (actual time=0.036..7.63 rows=3597 loops=1)
       -> Filter: (('EVENT'.EventDate >= DATE'2025-11-01') and ('EVENT'.VenueId is not null)) (cost=357 rows=1110) (actual time=0.0275..2.15 rows=3597 loops=1)
       -> Table scan on EVENT (cost=357 rows=3330) (actual time=0.0261..1.66 rows=3919 loops=1)
       -> Single-row index lookup on VENUE using PRIMARY (VenueId='EVENT'.VenueId) (cost=0.25 rows=1) (actual time=0.00133..0.00136 rows=1 loops=3597)

```

(4) CREATE INDEX venue_zip_idx ON VENUE (Zipcode); cost=7,122

```

1 -> Limit: 15 row(s) (cost=7122 rows=15) (actual time=28.3..28.9 rows=15 loops=1)
   -> Nested loop inner join (cost=7122 rows=22383) (actual time=28.3..28.9 rows=15 loops=1)
     -> Nested loop inner join (cost=968 rows=3330) (actual time=8.96..9.35 rows=170 loops=1)
       -> Sort: e.EventDate, e.EventTime, e.EventName (cost=357 rows=3330) (actual time=8.92..8.95 rows=170 loops=1)
       -> Filter: ((e.EventDate >= DATE'2025-11-01') and (e.VenueId is not null)) (cost=357 rows=3330) (actual time=0.0927..2.43 rows=3597 loops=1)
       -> Table scan on e (cost=357 rows=3330) (actual time=0.0852..1.98 rows=3919 loops=1)
       -> Filter: (v.Zipcode is not null) (cost=0.25 rows=1) (actual time=0.00211..0.00219 rows=1 loops=170)
     -> Single-row index lookup on v using PRIMARY (VenueId=e.VenueId) (cost=0.25 rows=1) (actual time=0.00197..0.002 rows=1 loops=170)
   -> Filter: <not>((z.Event_Count < <max>(select #3))) (cost=0.375..2.52 rows=6.72) (actual time=0.115..0.115 rows=0.0882 loops=170)
   -> Index lookup on z using <auto_key0> (Zipcode=v.Zipcode) (cost=0.25..2.52 rows=10.1) (actual time=0.0584..0.0587 rows=1 loops=170)
   -> Materialize (cost=0..0 rows=0) (actual time=9.74..9.74 rows=96 loops=1)
   -> Table scan on <temporary> (actual time=9.56..9.58 rows=96 loops=1)
   -> Aggregate using temporary table (actual time=9.56..9.56 rows=96 loops=1)
     -> Nested loop inner join (cost=746 rows=1110) (actual time=0.148..8.12 rows=3597 loops=1)
       -> Filter: (('EVENT'.EventDate >= DATE'2025-11-01') and ('EVENT'.VenueId is not null)) (cost=357 rows=1110) (actual time=0.132..2.37 rows=3597 loops=1)
       -> Table scan on EVENT (cost=357 rows=3330) (actual time=0.127..1.85 rows=3919 loops=1)
       -> Single-row index lookup on VENUE using PRIMARY (VenueId='EVENT'.VenueId) (cost=0.25 rows=1) (actual time=0.0014..0.00142 rows=1 loops=3597)
     -> Select #3 (subquery in condition; run only once)
     -> Table scan on <temporary> (actual time=9.48..9.49 rows=96 loops=1)
     -> Aggregate using temporary table (actual time=9.48..9.48 rows=96 loops=1)
     -> Nested loop inner join (cost=746 rows=1110) (actual time=0.0506..8.03 rows=3597 loops=1)
       -> Filter: (('EVENT'.EventDate >= DATE'2025-11-01') and ('EVENT'.VenueId is not null)) (cost=357 rows=1110) (actual time=0.041..2.21 rows=3597 loops=1)
       -> Table scan on EVENT (cost=357 rows=3330) (actual time=0.0389..1.69 rows=3919 loops=1)
       -> Single-row index lookup on VENUE using PRIMARY (VenueId='EVENT'.VenueId) (cost=0.25 rows=1) (actual time=0.00141..0.00144 rows=1 loops=3597)

```

Analysis:

The baseline query cost was 7,122. After creating the EVENT(EventDate) index, the query cost increased to 17,046, showing a clear decline in query efficiency. In contrast, creating indexes on EVENT(EventTime) and VENUE(Zipcode) resulted in no change of the original cost, indicating little to no improvement in performance. The increase in cost from the EVENT(EventDate) index suggests that MYSQL's optimizer found it less efficient to use this index for filtering, likely due to the small dataset and the use of many subqueries and aggregation. Since much of the query's runtime is spent on building temporary tables and computing grouped values, adding indexes on repeated scanned columns may not provide that much benefit.

The VENUE(Zipcode) index is the most strategic option for supporting future data growth. It directly supports the grouping and filtering logic on Zipcode in the main query as well as the subqueries; This would become more impactful as the dataset grows. While none of the indexes produced cost reductions in this current instance, indexing by Zipcode offers the best structural advantage for future query optimization.

